

Livrable UML complet base uniquement sur le code reel du projet : acteurs, donnees, classes et sources PlantUML / Mermaid reutilisables.

Date de generation 09/05/2026 16:19	Sources analysees backend/ + frontend/
Livrables PDF, HTML, SVG, PlantUML, Mermaid	Perimetre Routes, roles, models, migrations, controllers, services, policies, requests et ecrans relies

Cette documentation n'invente aucune fonctionnalite. Les points ambigus ou incohérents entre frontend, backend ou schema sont explicitement marques **a confirmer**.

Sommaire

- 1. Perimetre et methode
- 2. Diagramme de cas d'utilisation UML
- 3. MCD - modele conceptuel de donnees
- 4. Diagramme de classes UML
- 5. Points de vigilance pour la soutenance

1. Perimetre et methode

Cette documentation UML a ete construite uniquement a partir du code reel present dans le depot ouvert dans VS Code. Les sources principales examinees sont `backend/routes/api.php` , les `Models` , les `Controllers` , les `Policies` , les `Requests` , les `Services` , les migrations SQL et le routage React du frontend.

- Les acteurs ont ete deduits des roles reels declares dans `App\Enums\UserRole` , `frontend/src/utils/rbac.js` , les routes API et les ecrans React.
- Le MCD a ete deduit de l'etat final du schema via les migrations, puis recoupe avec les relations Eloquent et les contraintes applicatives.
- Le diagramme de classes se concentre sur les classes qui portent la logique metier, les dependances structurantes et les points d'autorisation.
- Quand une information n'est pas entierement garantie par la base mais seulement par le code applicatif, elle est marquee **a confirmer**.

Les schemas integres au PDF sont des visuels vectoriels lisibles en soutenance. Le code PlantUML / Mermaid exact est fourni juste apres chaque schema pour reutilisation.

2. Diagramme de cas d'utilisation UML

Le systeme expose six acteurs reels : `directeur` , `responsable` , `manager` , `technician` , `viewer` et `developer` . Les cas d'usage ci-dessous ne reprennent que des fonctions effectivement visibles dans les routes, policies, controllers et ecrans raccordes.

Acteur reel	Fonctionnalites confirmees par le code
<code>directeur</code>	Acteur global cote API. Passe tous les controles <code>role:*</code> . Peut gerer les operations metier, <code>users</code> , <code>settings</code> , <code>logs</code> et vider les tokens API.
<code>responsable</code>	Peut consulter le dashboard, les donnees metier, les notifications, gerer <code>clients</code> , <code>compteurs</code> , <code>secteurs</code> , <code>pannes</code> , <code>reparations</code> , <code>interventions</code> , <code>releves</code> et exporter les rapports.
<code>manager</code>	Peut consulter le dashboard, les donnees metier, les notifications, gerer <code>compteurs</code> , <code>pannes</code> , <code>reparations</code> , <code>interventions</code> , <code>releves</code> et exporter les rapports.
<code>technician</code>	Peut consulter son dashboard, les notifications, les donnees scopees par affectation, mettre a jour ses <code>pannes</code> assignees et gerer <code>reparations</code> , <code>interventions</code> , <code>releves</code> .
<code>viewer</code>	Lecture seule : dashboard, notifications et consultation des ressources metier.
<code>developer</code>	Role technique non production. Le code lui donne un comportement proche du <code>responsable</code> , sans les routes d'administration <code>directeur</code> .

Diagramme de cas d'utilisation - code reel

Vue de synthese. Le detail exact par ressource est precise dans les tableaux et dans le code PlantUML.

Systeme : RADEETA-Management-System

Se connecterSe deconnecter

Consulter tableau de bordConsulter notificationsConsulter donnees metierGerer `clients`Gerer `compteurs`Gerer `secteurs`Gerer `pannes`Gerer `reparations`Gerer `interventions`Gerer `releves`Exporter rapports PDF / ExcelGerer `users` / `settings` `logs` / tokens API

directeurresponsabledevelopermanagertechnicianviewer

A confirmer :

le frontend declare `viewer/reports`,

mais `reports/\*` est protege cote API

par `managerRoles`.

Figure 1 - Diagramme de cas d'utilisation de synthese. Les regroupements visuels sont affines par le tableau des fonctionnalites et le code PlantUML ci-dessous.

Code PlantUML pret a copier

```
@startuml
left to right direction
skinparam packageStyle rectangle
|
actor "directeur" as Directeur
actor "responsable" as Responsable
actor "manager" as Manager
actor "technician" as Technician
actor "viewer" as Viewer
actor "developer" as Developer
|
rectangle "RADEETA-Management-System" {
    usecase "Se connecter /\nSe deconnecter" as UC_Auth
    usecase "Consulter tableau de bord" as UC_Dashboard
    usecase "Consulter donnees metier" as UC_ReadData
    usecase "Consulter notifications" as UC_Notifications
    usecase "Gerer `clients`" as UC_Clients
    usecase "Gerer `compteurs`" as UC_Compteurs
    usecase "Gerer `secteurs`" as UC_Secteurs
    usecase "Gerer `pannes`" as UC_Pannes
    usecase "Gerer `reparations`" as UC_Reparations
    usecase "Gerer `interventions`" as UC_Interventions
    usecase "Gerer `releves`" as UC_Releves
    usecase "Exporter rapports PDF / Excel" as UC_Reports
    usecase "Gerer `users` /\n`settings` /\n`logs` /\ntokens API" as UC_Admin
}
}
```

```
|
Directeur --> UC_Auth
Directeur --> UC_Dashboard
Directeur --> UC_ReadData
Directeur --> UC_Notifications
Directeur --> UC_Clients
Directeur --> UC_Compteurs
Directeur --> UC_Secteurs
Directeur --> UC_Pannes
Directeur --> UC_Reparations
Directeur --> UC_Interventions
Directeur --> UC_Relevés
Directeur --> UC_Reports
Directeur --> UC_Admin
|
```

```
Responsable --> UC_Auth
Responsable --> UC_Dashboard
Responsable --> UC_ReadData
Responsable --> UC_Notifications
Responsable --> UC_Clients
Responsable --> UC_Compteurs
Responsable --> UC_Secteurs
Responsable --> UC_Pannes
Responsable --> UC_Reparations
Responsable --> UC_Interventions
Responsable --> UC_Relevés
Responsable --> UC_Reports
|
```

```
Manager --> UC_Auth
Manager --> UC_Dashboard
Manager --> UC_ReadData
```

```
Manager --> UC_Notifications
Manager --> UC_Compteurs
Manager --> UC_Pannes
Manager --> UC_Reparations
Manager --> UC_Interventions
Manager --> UC_Relevés
Manager --> UC_Reports
|
Technician --> UC_Auth
Technician --> UC_Dashboard
Technician --> UC_ReadData
Technician --> UC_Notifications
Technician --> UC_Pannes
Technician --> UC_Reparations
Technician --> UC_Interventions
Technician --> UC_Relevés
|
Viewer --> UC_Auth
Viewer --> UC_Dashboard
Viewer --> UC_ReadData
Viewer --> UC_Notifications
|
Developer --> UC_Auth
Developer --> UC_Dashboard
Developer --> UC_ReadData
Developer --> UC_Notifications
Developer --> UC_Clients
Developer --> UC_Compteurs
Developer --> UC_Secteurs
Developer --> UC_Pannes
Developer --> UC_Reparations
```

```

Developer --> UC_Interventions
Developer --> UC_Relevés
Developer --> UC_Reports
|
note right of UC_Admin
    Seulement `directeur`
    dans les routes API.
end note
|
note bottom of UC_Reports
    `viewer/reports` existe cote frontend
    mais l'API `reports/*` est a confirmer
    pour `viewer`.
end note
@enduml

```

Source : [docs/uml-assets/use-case-diagram.puml](#) .

Explication courte

Le code montre un systeme centre sur la gestion operationnelle. `directeur` est un super-role cote API. `responsable` couvre presque tout le metier. `manager` supervise sans suppression globale. `technician` agit surtout sur les elements assignes. `viewer` reste en lecture seule. `developer` est un role technique reserve au non-production.

A confirmer : le frontend declare une route `/viewer/reports` , mais l'API `reports/*` est protegee par `managerRoles` dans `backend/routes/api.php` . Le role `viewer` n'est donc pas documente comme ayant un acces confirme aux exports.

3. MCD - modele conceptuel de donnees

Le MCD ci-dessous represente les entites les plus importantes du projet. Les tables techniques annexes de Laravel n'ont pas ete detaillees, sauf `notifications` car elles sont effectivement utilisees par le code applicatif.

Entite	Attributs importants	Role dans le domaine
<code>users</code>	<code>id</code> , <code>nom</code> , <code>prenom</code> , <code>identifiant</code> , <code>email</code> , <code>agence</code> , <code>role</code>	Affecte des <code>pannes</code> , execute des <code>reparations</code> , cree des <code>relevés</code> , porte des <code>interventions</code> , des <code>activity_logs</code> et des <code>notifications</code> .
<code>secteurs</code>	<code>id</code> , <code>nom_secteur</code> , <code>emplacement</code> , <code>agence</code> , <code>num_torne</code> , <code>latitude</code> , <code>longitude</code>	Reference géographique pour <code>clients</code> et <code>compteurs</code> .
<code>clients</code>	<code>id</code> , <code>police</code> , <code>nom</code> , <code>prenom</code> , <code>cin</code> , <code>telephone</code> , <code>adresse</code> , <code>type_abonnement</code> , <code>service_type</code> , <code>abonne</code> , <code>id_secteur</code>	Un client possede des <code>compteurs</code> et peut etre rattache indirectement aux <code>interventions</code> .
<code>compteurs</code>	<code>id</code> , <code>cadran</code> , <code>calibre</code> , <code>marque</code> , <code>service_type</code> , <code>index_releve</code> , <code>id_client</code> , <code>id_secteur</code>	Point central du domaine : lie <code>clients</code> , <code>secteurs</code> , <code>pannes</code> , <code>relevés</code> et <code>interventions</code> .
<code>pannes</code>	<code>id</code> , <code>id_compteur</code> , <code>date_panne</code> , <code>anomalie</code> , <code>status</code> , <code>assigned_to</code>	Incident lie a un compteur et eventuellement assigne a un <code>User</code> .
<code>reparations</code>	<code>id</code> , <code>id_panne</code> , <code>id_plombier</code> , <code>date_reparation</code> , <code>description</code>	Trace une intervention de reparation sur une panne.
<code>relevés</code>	<code>id</code> , <code>compteur_id</code> , <code>ancien_index</code> , <code>nouvel_index</code> , <code>consommation</code> , <code>periode_debut</code> , <code>periode_fin</code> , <code>created_by</code>	Lecture de compteur avec consommation calculee.
<code>interventions</code>	<code>id</code> , <code>intervention_number</code> , <code>panne_id</code> , <code>client_id</code> , <code>meter_id</code> , <code>technician_id</code> , <code>service_type</code> , <code>work_type</code> , <code>priority</code> , <code>status</code> , <code>started_at</code> , <code>completed_at</code>	Intervention terrain rattachee a une panne, un client, un compteur et un technicien.

Entite	Attributs importants	Role dans le domaine
activity_logs	id , user_id , action , module , ip_address , metadata	Journalise les actions sensibles ou administratives.
settings	id , key , value	Stocke les parametres globaux de l'application.
notifications	id , type , notifiable_type , notifiable_id , data , read_at	Table Laravel actuelle pour les notifications en base. App\Models\Notification est a confirmer.

Association et cardinalites	Justification dans le code
secteurs 0,N - clients 0,1	FK clients.id_secteur , nullable.
secteurs 0,N - compteurs 1,1	FK compteurs.id_secteur , non nullable.
clients 0,N - compteurs 1,1	FK compteurs.id_client , non nullable.
compteurs 0,N - pannes 1,1	FK pannes.id_compteur , non nullable.
users 0,N - pannes 0,1	FK pannes.assigned_to , nullable.
pannes 0,N - reparations 1,1	Schema / modele: hasMany ; metier actif: a confirmer vers 0,1 a cause de RepairRequest .
users 0,N - reparations 0,1	FK reparations.id_plombier , nullable.
compteurs 0,N - releves 1,1	FK releves.compteur_id , non nullable.
users 0,N - releves 0,1	FK releves.created_by , nullable.
pannes 0,N - interventions 0,1	FK interventions.panne_id , nullable.

erDiagram

users {

bigint id PK

string nom

string prenom

string identifiant

string email

string agence

string role

}

secteurs {

bigint id PK

string nom_secteur

string emplacement

string agence

string num_torne

decimal latitude

decimal longitude

}

clients {

bigint id PK

string police

string nom

string prenom

string cin

string telephone

string adresse

string type_abonnement

```

    string service_type
    bigint id_secteur FK
    boolean abonne
}

|
compteurs {
    bigint id PK
    string cadran
    string calibre
    string marque
    string service_type
    decimal index_releve
    bigint id_client FK
    bigint id_secteur FK
}

|
pannes {
    bigint id PK
    bigint id_compteur FK
    date date_panne
    string anomalie
    string status
    bigint assigned_to FK
}

|
reparations {
    bigint id PK
    bigint id_panne FK
    bigint id_plombier FK
    date date_reparation
    text description

```

```

    }
|
    releves {
        bigint id PK
        bigint compteur_id FK
        decimal ancien_index
        decimal nouvel_index
        decimal consommation
        date periode_debut
        date periode_fin
        bigint created_by FK
    }
|
    interventions {
        bigint id PK
        string intervention_number
        bigint panne_id FK
        bigint client_id FK
        bigint meter_id FK
        bigint technician_id FK
        string service_type
        string work_type
        json materials_used
        text observations
        string priority
        string status
        datetime started_at
        datetime completed_at
    }
|
    activity_logs {

```

```

    bigint id PK
    bigint user_id FK
    string action
    string module
    string ip_address
    json metadata
}

|
settings {
    bigint id PK
    string key
    json value
}

|
notifications {
    uuid id PK
    string type
    string notifiable_type
    bigint notifiable_id
    json data
    datetime read_at
}

|
secteurs o|--o{ clients : "id_secteur"
secteurs ||--o{ compteurs : "id_secteur"
clients ||--o{ compteurs : "id_client"
compteurs ||--o{ pannes : "id_compteur"
users o|--o{ pannes : "assigned_to"
pannes ||--o{ reparations : "id_panne"
users o|--o{ reparations : "id_plombier"
compteurs ||--o{ releves : "compteur_id"

```

```
users o|--o{ releves : "created_by"
pannes o|--o{ interventions : "panne_id"
users ||--o{ interventions : "technician_id"
clients o|--o{ interventions : "client_id"
compteurs o|--o{ interventions : "meter_id"
users o|--o{ activity_logs : "user_id"
users ||--o{ notifications : "notifiable_id (usage actuel)"
```

Source : `docs/uml-assets/mcd.mmd` .

Explication courte

Le coeur de donnees est organise autour de `Compteur` , relie a `Client` , `Secteur` , `Panne` , `Releve` et `Intervention` . `User` intervient comme acteur assignable, createur ou technicien. `Setting` , `ActivityLog` et `notifications` portent la dimension systeme et administrative.

A confirmer : `App\Models\Notification` ne correspond pas a la table Laravel `notifications` actuellement utilisee par `NotificationController` , qui passe par `Illuminate\Notifications\DatabaseNotification` .

4. Diagramme de classes UML

Le diagramme de classes met en avant les modeles metier, les classes de regles et quelques controllers structurants. Les dependances ont ete reconstruites a partir des injections, des appels directs et des relations Eloquent.

Classe principale	Role	Attributs / methodes importants
User	Modele d'authentification et de role	Attributs clefs : <code>identifiant</code> , <code>agence</code> , <code>role</code> . Methodes : <code>reparations()</code> , <code>interventions()</code> , <code>assignedPannes()</code> .
Secteur	Modele geographique	Methodes : <code>compteurs()</code> , <code>pannes()</code> .
Client	Modele abonne	Methodes : <code>compteurs()</code> , <code>secteur()</code> .
Compteur	Modele central de rattachement	Methodes : <code>client()</code> , <code>secteur()</code> , <code>pannes()</code> , <code>releves()</code> .
Panne	Incident metier	Methodes : <code>compteur()</code> , <code>reparations()</code> , <code>interventions()</code> , <code>assignedOperator()</code> .
Reparation	Action de reparation	Methodes : <code>panne()</code> , <code>plombier()</code> .
Releve	Lecture de compteur	Methodes : <code>compteur()</code> , <code>creator()</code> . Regle metier : consommation calculee dans <code>booted()</code> .
Intervention	Intervention terrain	Methodes : <code>panne()</code> , <code>technician()</code> , <code>client()</code> , <code>meter()</code> . Regle metier : numero auto dans <code>creating()</code> .
Setting	Parametrage global	Attributs : <code>key</code> , <code>value</code> .
ActivityLog	Journalisation	Methode statique : <code>record(...)</code> .
NotificationService	Service de diffusion des notifications	Methodes : <code>panneAssigned()</code> , <code>repairCompleted()</code> , <code>userCreated()</code> .

Classe principale	Role	Attributs / methodes importants
OperatorAccess	Filtrage metier pour technician	Methodes de scope : scopePannes() , scopeReparations() , scopeRelevés() , scopeCompteurs() , scopeClients() .
InterventionPolicy	Regles d'autorisation sur les interventions	Methodes : viewAny() , view() , create() , update() .
ReparationRequest	Validation metier sur les reparations	Methodes : authorize() , rules() , withValidator() .
AuthController	Authentification API	Methodes : login() , logout() , me() , clearTokens() .
PanneController	CRUD des pannes	Depend de NotificationService et applique une logique specifique a technician .
ReparationController	CRUD des reparations	Depend de ReparationRequest , NotificationService , OperatorAccess , Panne .
InterventionController	CRUD des interventions	Depend de InterventionPolicy , OperatorAccess , ActivityLog , Panne .

Diagramme de classes UML - synthese du code

La visualisation ci-dessous se concentre sur les classes metier et les regles. Le code PlantUML complet est fourni ensuite. Domaine et persistence Services / regles Controllers

Authenticatable + login identity **User** + nom, prenom + identifiant, email + agence, role + reparations() + interventions() + assignedPannes() **Secteur** + nom_secteur + emplacement + agence, num_torne + latitude, longitude + compteurs() + pannes() **Client** + police + nom, prenom + cin, telephone + type_abonnement + service_type, abonne + compteurs() + secteur() **Compteur** + cadran + calibre, marque + service_type + index_releve + client() + secteur() + pannes(), releves() **Panne** + date_panne + anomalie + status + assigned_to + compteur() + reparations() + interventions() **Setting** + key + value **ActivityLog** + action, module + ip_address + metadata + user() + record(...) **Reparation** + id_panne + id_plombier + date_reparation + description + panne(), plombier() **Releve** + compteur_id + ancien_index, nouvel_index + consommation + periode_debut / fin + compteur() + creator() **Intervention** + intervention_number + panne_id, client_id, meter_id + technician_id + service_type, work_type + priority, status + started_at, completed_at + panne(), technician() + client(), meter() **NotificationService** + panneAssigned() + repairCompleted() + userCreated() **OperatorAccess** + isOperator() + scopePannes() + scopeReparations() + scopeRelevés() + scopeCompteurs() **InterventionPolicy** + viewAny() + view(), create() + update() **ReparationRequest** + authorize() + rules() + withValidator() **AuthController** + login() + logout() + me(), clearTokens() **PanneController** + index(), store() + show(), update() + destroy()

ReparationController + index(), store() + show(), update() + destroy() **InterventionController** + index(), store() + show(), update() + validated() Autres classes presentes dans le code

PlantUML : ClientController ReleveController UserController SettingsController NotificationController

association dependance heritage

Figure 3 - Diagramme de classes de synthese. Le code PlantUML complet detaille un ensemble plus large de controllers et de dependances.

Code PlantUML pret a copier

```
@startuml
left to right direction
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
hide empty members
|
abstract class Authenticatable
abstract class Model
abstract class Controller
|
package "Models" {
class User {
+id: bigint
+nom: string
+prenom: string?
+identifiant: string
+email: string?
+agence: string
+role: UserRole
+reparations()
+interventions()
+assignedPannes()
}
}
```

```
|
class Secteur {
+id: bigint
+nom_secteur: string
+emplacement: string
+agence: string
+num_torne: string
+latitude: decimal?
+longitude: decimal?
+compteurs()
+pannes()
}

|
class Client {
+id: bigint
+police: string
+nom: string
+prenom: string?
+cin: string?
+telephone: string?
+adresse: string?
+type_abonnement: string
+service_type: string
+id_secteur: bigint?
+abonne: bool
+compteurs()
+secteur()
}

|
class Compteur {
+id: bigint
```

```

+cadran: string
+calibre: string
+marque: string?
+service_type: string
+index_releve: decimal
+id_client: bigint
+id_secteur: bigint
+client()
+secteur()
+pannes()
+releves()
}

class Panne {
+id: bigint
+id_compteur: bigint
+date_panne: date
+anomalie: PanneAnomalie
+status: PanneStatus
+assigned_to: bigint?
+compteur()
+reparations()
+interventions()
+assignedOperator()
}

class Reparation {
+id: bigint
+id_panne: bigint
+id_plombier: bigint?
+date_reparation: date

```

```

+description: text?
+panne()
+plombier()
}

class Releve {
+id: bigint
+compteur_id: bigint
+ancien_index: decimal
+nouvel_index: decimal
+consommation: decimal
+periode_debut: date
+periode_fin: date
+created_by: bigint?
+compteur()
+creator()
}

class Intervention {
+id: bigint
+intervention_number: string
+panne_id: bigint?
+client_id: bigint?
+meter_id: bigint?
+technician_id: bigint
+service_type: string
+work_type: string
+materials_used: array?
+observations: text?
+priority: string
+status: string

```

```

+started_at: datetime
+completed_at: datetime?
+panne()
+technician()
+client()
+meter()
}

class Setting {
+id: bigint
+key: string
+value: json
}

class ActivityLog {
+id: bigint
+user_id: bigint?
+action: string
+module: string
+ip_address: string?
+metadata: json?
+user()
+record(action, module, request, metadata)
}

class DatabaseNotification {
+id: uuid
+type: string
+data: json
+read_at: datetime?
+created_at: datetime?
}

```

```

    }
}

package "Services / Policies / Requests" {
    class NotificationService {
        +panneAssigned(panne)
        +repairCompleted(reparation)
        +userCreated(user)
    }

    class OperatorAccess {
        +isOperator(user)
        +scopePannes(query, user)
        +scopeReparations(query, user)
        +scopeRelevés(query, user)
        +scopeCompteurs(query, user)
        +scopeClients(query, user)
        +canAccessCompteur(compteur, user)
    }

    class InterventionPolicy {
        +viewAny(user)
        +view(user, intervention)
        +create(user)
        +update(user, intervention)
    }

    class ReparationRequest {
        +authorize()
        +rules()
        +withValidator()
    }
}

```

```
}  
}  
|  
package "Controllers" {  
  class AuthController {  
    +login(request)  
    +logout(request)  
    +me(request)  
    +clearTokens()  
  }  
|  
  class DashboardController {  
    +summary()  
  }  
|  
  class NotificationController {  
    +index(request)  
    +markAsRead(request)  
  }  
|  
  class ClientController {  
    +index(request)  
    +store(request)  
    +show(request, client)  
    +update(request, client)  
    +destroy(client)  
  }  
|  
  class PanneController {  
    +index(request)  
    +store(request, notifications)
```



```

+show(request, panne)
+update(request, panne, notifications)
+destroy(panne)
}

class ReparationController {
+index(request)
+store(request, notifications)
+show(request, reparation)
+update(request, reparation, notifications)
+destroy(reparation)
}

class ReleveController {
+index(request)
+store(request)
+update(request, releve)
+show(request, releve)
}

class InterventionController {
+index(request, policy)
+store(request, policy)
+show(request, intervention, policy)
+update(request, intervention, policy)
}

class UserController {
+index(request)
+technicians()
+store(request, notifications)

```

```

    +update(request, user)
}

class SettingsController {
    +show()
    +update(request)
}

class ReportController {
    +pannesPdf(request)
    +clientsExcel()
}
}

User --|> Authenticatable
Secteur --|> Model
Client --|> Model
Compteur --|> Model
Panne --|> Model
Reparation --|> Model
Releve --|> Model
Intervention --|> Model
Setting --|> Model
ActivityLog --|> Model

AuthController --|> Controller
DashboardController --|> Controller
NotificationController --|> Controller
ClientController --|> Controller
PanneController --|> Controller
ReparationController --|> Controller

```

```

ReleveController --|> Controller
InterventionController --|> Controller
UserController --|> Controller
SettingsController --|> Controller
ReportController --|> Controller
|
Client "0..*" --> "0..1" Secteur : secteur
Secteur "1" <-- "0..*" Compteur : compteurs
Client "1" <-- "0..*" Compteur : client
Compteur "1" <-- "0..*" Panne : compteur
Panne "0..*" --> "0..1" User : assignedOperator
Panne "1" <-- "0..*" Reparation : panne
Reparation "0..*" --> "0..1" User : plombier
Compteur "1" <-- "0..*" Releve : compteur
Releve "0..*" --> "0..1" User : creator
Panne "0..1" <-- "0..*" Intervention : panne
User "1" <-- "0..*" Intervention : technician
Client "0..1" <-- "0..*" Intervention : client
Compteur "0..1" <-- "0..*" Intervention : meter
User "0..1" <-- "0..*" ActivityLog : user
User "1" <-- "0..*" DatabaseNotification : notifications
|
NotificationService ..> Panne
NotificationService ..> Reparation
NotificationService ..> User
NotificationService ..> DatabaseNotification
|
OperatorAccess ..> User
OperatorAccess ..> Compteur
OperatorAccess ..> Panne
OperatorAccess ..> Reparation

```

```
OperatorAccess ..> Releve
OperatorAccess ..> Client
|
InterventionPolicy ..> User
InterventionPolicy ..> Intervention
|
ReparationRequest ..> Panne
ReparationRequest ..> Reparation
ReparationRequest ..> User
|
AuthController ..> User
AuthController ..> ActivityLog
DashboardController ..> Client
DashboardController ..> Compteur
DashboardController ..> Secteur
DashboardController ..> Panne
DashboardController ..> Intervention
NotificationController ..> DatabaseNotification
NotificationController ..> User
ClientController ..> Client
ClientController ..> Secteur
ClientController ..> OperatorAccess
PanneController ..> Panne
PanneController ..> NotificationService
ReparationController ..> Reparation
ReparationController ..> Panne
ReparationController ..> ReparationRequest
ReparationController ..> NotificationService
ReparationController ..> OperatorAccess
ReleveController ..> Releve
ReleveController ..> Compteur
```

```

ReleveController ..> OperatorAccess
InterventionController ..> Intervention
InterventionController ..> Panne
InterventionController ..> ActivityLog
InterventionController ..> InterventionPolicy
InterventionController ..> OperatorAccess
UserController ..> User
UserController ..> NotificationService
UserController ..> ActivityLog
SettingsController ..> Setting
SettingsController ..> ActivityLog
ReportController ..> Panne
|
note bottom of ReparationRequest
    Une seule reparation active par panne
    est imposee par validation applicative,
    pas par une contrainte SQL unique.
end note
|
note bottom of DatabaseNotification
    Table Laravel `notifications`.
    `App\Models\Notification` est legacy
    et reste a confirmer.
end note
@enduml

```

Source : `docs/uml-assets/class-diagram.puml` .

Explication courte

Le domaine applicatif est porte par les modeles Laravel. Les regles d'autorisation et de filtrage sont centralisees dans `InterventionPolicy` , `OperatorAccess` et `ReparationRequest` . Les controllers orchestrent les cas d'usage API sans dupliquer la logique de persistance.

5. Points de vigilance pour la soutenance

- Le schema SQL autorise plusieurs `reparations` pour une meme `panne` , mais la validation applicative en limite une seule active a la fois.
- Le role `developer` existe reellement dans le code, mais il est explicitement bloque en production dans `AuthController` , `EnsureUserHasRole` et `rbac.js` .
- Le projet contient des traces legacy ou paralleles, mais les diagrammes ci-dessus n'utilisent que les classes, routes et ecrans reels relies au comportement actif du systeme.
- Les notifications visibles par l'application s'appuient sur la table Laravel `notifications` et non sur le modele local `App\Models\Notification` , qui reste a confirmer.

Le livrable final est pret pour une soutenance : il fournit une vue metier, une vue donnees, une vue classes et les sources reutilisables des diagrammes. Les fichiers auxiliaires ont ete generes dans `docs/uml-assets` pour faciliter la reprise future.